COURSE DL201 / MODULE DL

Training Deep Neural Networks

Learning goal: explain core concepts, compare related methods, and apply them to realistic machine learning or educational technology scenarios.

Document ID: DL201-02_Deep_Learning_Training | Format: instructional Q&A | Revision: 1.0

01. What is mini-batch training?

Mini-batch training computes an update from a subset of examples, trading memory use and gradient noise against speed. An epoch processes the training dataset once. Shuffle examples when order does not carry meaning. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

02. How do SGD and Adam differ?

SGD updates weights using the current batch gradient, often with momentum. Adam adapts each parameter's step using running estimates of gradient moments. Adam is a useful default, but validation should decide optimizer and learning rate. A useful way to understand how do sgd and adam differ is to compare the methods not only by definition, but also by the kind of input they expect, the output they produce, and the situations in which each is useful. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

03. What causes vanishing and exploding gradients?

Repeated multiplication through deep or recurrent layers can shrink or amplify gradients. Vanishing slows learning in early layers; exploding can destabilize updates. Residual connections, suitable initialization, gating, and gradient clipping can help. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

04. What is weight initialization?

Initialization sets starting parameter values so signals and gradients have a useful scale. Poor initialization can saturate activations or prevent symmetry breaking. Xavier and He approaches account for layer width and common activation choices. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

05. How does batch normalization help?

Batch normalization normalizes intermediate activations using batch statistics during training, then stored running statistics at inference. It can stabilize optimization, but small batches or mismatched training and inference conditions need care. In practice, it helps to think of this as a sequence of steps rather than a single operation. Each step affects the information available to the next stage, so implementation choices should be validated on representative data. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

06. What are dropout and weight decay?

Dropout randomly masks activations during training to reduce reliance on individual units; inference uses the full network. Weight decay discourages large weights through an optimizer penalty. Both may reduce overfitting when tuned appropriately. This concept is important because it connects the underlying theory with decisions made when building a real system. Understanding both its purpose and its limitations helps avoid treating the technique as a rule that works equally well in every dataset. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.

07. When should early stopping be used?

Track a validation metric and stop when improvement stalls for a chosen patience. Restore the best saved weights rather than the final epoch. The test set remains untouched until model selection is complete. The choice is context dependent. The right approach depends on the objective, the data distribution, the cost of errors, and operational constraints such as latency, memory, interpretability, and the amount of labeled data available. For example, in an educational technology system, the same idea can be tested using course content, student questions, assessment records, or retrieved passages. A team should define the expected behavior first, measure it on held-out examples, and inspect failures rather than relying only on an aggregate score. For a production or RAG-oriented workflow, preserve useful metadata and reproducible preprocessing settings so that an unexpected result can be traced back to its source. This also makes it easier to compare alternative configurations and determine whether an apparent improvement comes from better modeling, better retrieval, or simply a change in the evaluation data. An important limitation is that no single technique or metric guarantees good real-world performance. Data leakage, distribution shift, poorly chosen evaluation examples, and mismatched assumptions can all make results look stronger than they really are. The practical approach is therefore to establish a baseline, change one major factor at a time, and verify the effect using representative validation data.